# LangChain, LlamaIndex and Haystack: A Practical Guide

Nov 3, 2026 · @Rajnish Maurya

## Overview

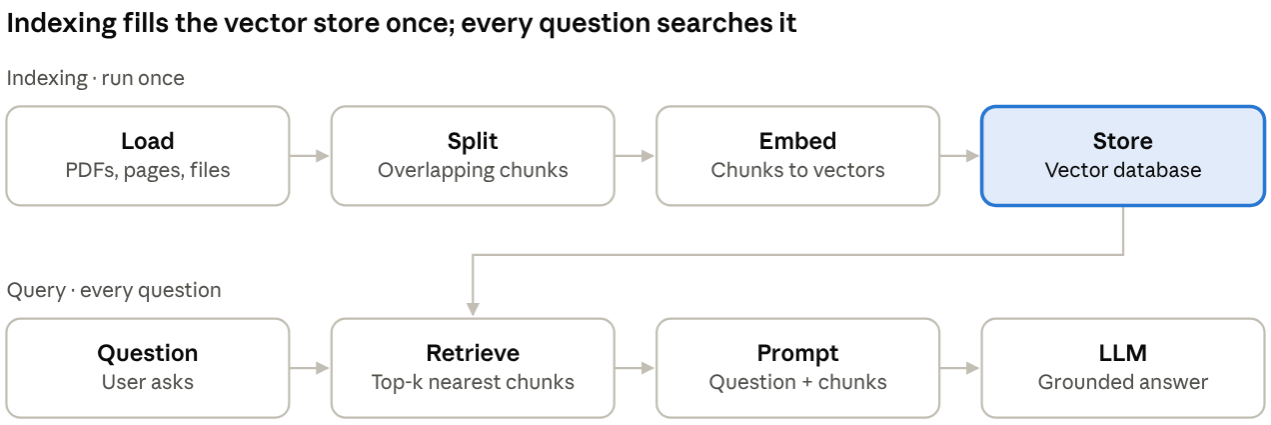
LangChain, LlamaIndex and Haystack are open-source Python frameworks for building applications on top of large language models (LLMs), and all three now handle both retrieval-augmented generation (RAG) and tool-using agents. They differ mainly in what they put at the centre: LangChain is the broadest general toolkit, LlamaIndex is built around getting documents indexed and retrieved well, and Haystack is built around explicit, typed pipelines.

A model on its own only takes text in and returns text. These frameworks supply the surrounding pieces a real application needs: loading and chunking documents, embedding and storing them, retrieving the right passages, calling tools, keeping conversation state, and swapping model providers without rewriting code.

| Framework | Maker | First released | Centre of gravity |
|---|---|---|---|
| LangChain | LangChain Inc. (Harrison Chase) | 2022 | General LLM toolkit and agents, via LangGraph |
| LlamaIndex | LlamaIndex Inc. (Jerry Liu) | Late 2022 (as GPT Index) | Data ingestion, indexing and retrieval |
| Haystack | deepset (Berlin) | 2019 | Typed component pipelines for search and RAG |

## RAG in one picture

All three frameworks implement the same two-phase RAG pipeline; they differ in how much of it they hide and what they call each step.




RAG pipeline · indexing and query phases

The vector store is the bridge: indexing fills it, and every question searches it with the same embedding model.

| Step | LangChain | LlamaIndex | Haystack |
|---|---|---|---|
| Load | Document loaders | Readers (LlamaHub), LlamaParse | Converters |
| Split | Text splitters | Node parsers | DocumentSplitter |
| Embed and store | Embeddings + vector store | `VectorStoreIndex` | Document embedder + writer |
| Retrieve | Retriever | Retriever | Retriever component |
| Prompt and answer | Prompt template + chat model | Query engine | Prompt builder + generator |

# LangChain

LangChain is the most widely used of the three and the broadest: a Python and TypeScript library of standard building blocks for LLM apps, with hundreds of integrations for models, vector databases and tools. Its v1.0 release reached general availability on 22 October 2025, and its agents are now built on LangGraph.

## Core building blocks

- **Chat models:** one interface over OpenAI, Anthropic, Gemini, Hugging Face and local models, so changing provider is a one-string change.
- **Prompt templates and output parsers:** reusable prompts with variables, and conversion of model output into JSON or typed objects.

- **Document loaders and text splitters:** read PDFs, web pages, Notion and more into `Document` objects, then cut them into overlapping chunks.
- **Embeddings, vector stores and retrievers:** turn chunks into vectors, store them (Chroma, FAISS, Pinecone, pgvector, Qdrant) and fetch the closest ones for a question.
- **Tools and agents:** functions the model can choose to call, run in a loop until it has an answer.

## The ecosystem

- **LangChain** (`langchain`, `langchain-core`): the component library.
- **LangGraph:** a lower-level framework that models an agent as a graph of steps with state, branching, retries and human-approval checkpoints. It is the recommended way to build multi-step agents.
- **LangSmith:** a paid platform for tracing, debugging and evaluating LLM apps.

## Example: a minimal agent

```
from langchain.chat_models import init_chat_model
from langchain.agents import create_agent

def get_weather(city: str) -> str:
    """Get the weather for a city."""
    return f"It's sunny in {city}."

model = init_chat_model("anthropic:claude-sonnet-4-5")
agent = create_agent(model, tools=[get_weather])
agent.invoke({"messages": [{"role": "user", "content": "Weather in
Coimbatore?"}]})
```

## Strengths and weaknesses

- **Strengths:** largest integration catalogue and community; LangGraph gives fine control over complex agents; LangSmith is the most complete first-party observability tool among the three.
- **Weaknesses:** abstractions can feel heavy and make debugging harder than plain code; frequent API changes mean 2023-era tutorials are often broken; basic RAG needs more wiring than in LlamaIndex.

# LlamaIndex

LlamaIndex is a Python framework built around one job: turning your documents into something an LLM can search and reason over. It needs the least code for "chat with my documents" and has the deepest retrieval and parsing features of the three. It is now effectively Python-only: LlamaIndex.TS was deprecated in March 2026 and archived on 30 April 2026.

## Core concepts

- **Document:** a loaded file or record.
- **Node:** a chunk of a document, with its text, metadata and links to neighbouring chunks.
- **Index:** the structure built over the nodes; most often a `VectorStoreIndex`, with keyword, tree and knowledge-graph indexes also available.
- **Retriever:** fetches the relevant nodes for a query.
- **Query engine / chat engine:** retrieval plus the LLM as one question-in, answer-out object; the chat engine also remembers the conversation.

## Example: RAG in a few lines

```
from llama_index.core import VectorStoreIndex, SimpleDirectoryReader

docs = SimpleDirectoryReader("./handbook").load_data()   # load
index = VectorStoreIndex.from_documents(docs)             # split + embed +
store
query_engine = index.as_query_engine(similarity_top_k=4)  # retrieve + prompt
+ LLM

print(query_engine.query("How many leave days do new employees get?"))
```

Every default (splitter, embedding model, vector store, LLM) can be swapped, and the response carries the source nodes used, which makes citing pages simple.

## Beyond basic RAG

- **Advanced retrieval:** hierarchical chunking (match small chunks, return their larger parent), routers that pick an index per question, and sub-question engines that split a complex question across sources.
- **Parsing:** LlamaParse, a paid cloud service, extracts clean text from hard PDFs with tables, multi-column layouts and scans. LiteParse, released in 2026, is an open-source local parser that preserves layout.

- **Workflows and agents:** event-driven steps that listen for and emit events, plus prebuilt `FunctionAgent` and `ReActAgent` classes built on them. Workflows replaced Query Pipelines from version 0.11.

### Strengths and weaknesses

- **Strengths:** concise RAG with little boilerplate; best-in-class document parsing; rich retrieval strategies out of the box.
- **Weaknesses:** Python only; agent tooling is less mature than LangGraph; the best parsing and managed indexing (LlamaParse, LlamaCloud) are paid.

## Haystack

Haystack is deepset's Apache-2.0 Python framework that builds every application as an explicit graph of typed components. It is the oldest of the three, first released in 2019 for neural search and question answering, and favours pipelines that are easy to inspect, test, serialise and deploy.

### Core concepts

- **Component:** a class that does one job (embed text, retrieve documents, build a prompt, call an LLM) with declared, typed inputs and outputs.
- **Pipeline:** components wired together by name; mismatched connections fail when the pipeline is built, not mid-request. Pipelines can branch, loop and route.
- **Document store:** the vector or keyword database; an in-memory store ships for prototyping.
- **Converters and generators:** converters turn PDFs and HTML into documents; generators are the provider-specific LLM components.

## Example: the query pipeline

```
from haystack import Pipeline
from haystack.components.embedders import SentenceTransformersTextEmbedder
from haystack.components.retrievers.in_memory import
InMemoryEmbeddingRetriever
from haystack.components.builders import ChatPromptBuilder
from haystack.components.generators.chat import OpenAIChatGenerator

rag = Pipeline()
rag.add_component("embedder", SentenceTransformersTextEmbedder())
rag.add_component("retriever",
InMemoryEmbeddingRetriever(document_store=store, top_k=4))
rag.add_component("prompt", ChatPromptBuilder(template=template))
rag.add_component("llm", OpenAIChatGenerator())

rag.connect("embedder.embedding", "retriever.query_embedding")
rag.connect("retriever.documents", "prompt.documents")
rag.connect("prompt.prompt", "llm.messages")

result = rag.run({"embedder": {"text": q}, "prompt": {"question": q}})
```

Every RAG step is a named component and every arrow is a `connect` call. Indexing is a second pipeline of the same kind (converter, splitter, document embedder, writer). This example uses the 2.x API; check it against current docs.

## Production features and agents

- **YAML serialisation:** pipelines can be saved as configuration, versioned and reviewed like code.
- **Retrieval toolkit:** hybrid keyword plus vector search, rerankers and built-in pipeline evaluation.
- **deepset Studio:** a free visual builder that exports to Python or YAML; a paid enterprise platform adds prompt versioning and testing.
- **Agents:** an `Agent` is just another component, and a whole pipeline can be exposed as a tool. Version 3.0, released in July 2026, added hooks around the agent loop, built-in state such as step counts and token usage, skills, and ready-made agents such as deep research.

## Strengths and weaknesses

- **Strengths:** explicit, typed, inspectable pipelines; YAML deployment; strong classic search and evaluation features.

- **Weaknesses:** more boilerplate; smaller ecosystem and community than LangChain; switching models means replacing the generator component; Haystack 1.x code is incompatible with 2.x and later.

## Side-by-side comparison

LlamaIndex wins on document-heavy RAG, LangChain on breadth and complex agents, and Haystack on explicit, deployable pipelines.

| Dimension | LangChain | LlamaIndex | Haystack |
|---|---|---|---|
| Design philosophy | Toolkit of interchangeable building blocks | Index your data, then query it | Explicit graph of typed components |
| Languages | Python, TypeScript | Python (TypeScript archived 2026) | Python |
| Licence | MIT | MIT | Apache 2.0 |
| Code for basic RAG | Moderate | Least (about 3 lines) | Most (explicit wiring) |
| Retrieval depth | Good, assembled from parts | Deepest: routers, hierarchical chunks, sub-questions | Strong: hybrid search, rerankers |
| Document parsing | Many loaders, basic quality | LlamaParse (paid) and LiteParse (local) | Converter components |
| Agent framework | LangGraph (graph-based, most mature) | Workflows (event-driven) | Agent component inside pipelines |
| Config and deployment | Code-first; LangGraph platform | Code-first; LlamaCloud, llama-agents | YAML pipelines, deepset Studio |
| Observability | LangSmith (first-party, paid) | Third-party (Arize Phoenix, Langfuse) | Third-party; enterprise platform |
| Ecosystem size | Largest | Large | Smaller |
| Switching LLM provider | Change one model string | Change one setting | Replace the generator component |

## How to choose

Pick by what dominates your application: documents, agents, or maintainable pipelines.

| If your app is mainly… | Choose | Why |
|---|---|---|
| Q&A over contracts, manuals, papers, invoices | LlamaIndex | Least code, best parsing and retrieval |
| An agent calling many tools with multi-step logic | LangChain + LangGraph | Fine control, largest integration set, LangSmith tracing |
| Enterprise search or RAG a team maintains for years | Haystack | Typed, testable pipelines saved as YAML |
| A TypeScript or JavaScript project | LangChain | The only one of the three with a maintained JS version |
| A prototype to learn the concepts | LlamaIndex or LangChain | Most tutorials and fastest start |

**Mixing is common.** A typical pattern uses LlamaIndex for ingestion and retrieval, wrapped as a tool inside a LangGraph agent. The frameworks share concepts (documents, chunks, embeddings, retrievers), so ideas transfer even when code does not.

**Sometimes none is needed.** For a stable, simple workflow, a provider SDK plus a vector database client in plain Python can be easier to debug and maintain than any framework.

**Quality comes from indexing.** Whatever you choose, most RAG failures trace back to poor text extraction, chunks that are too large or too small, and missing metadata filters, not to the framework or the model.

## Sources


- What is LangChain? (Future AGI)
- What is LlamaIndex in 2026 (Future AGI)
- LlamaIndex in 2026: Workflows, Agents, and What Changed (Developers Digest)
- LlamaIndex organisation profile (AI Engineer)
- Haystack reference (Real Python)
- Haystack: deepset's open-source RAG framework (Noze)
- LangChain vs LlamaIndex (Addepto)